

OSSP (Operating Systems and Systems Programming) –25CS2104E

OSSP LAB – PRACTICAL-4

Aim

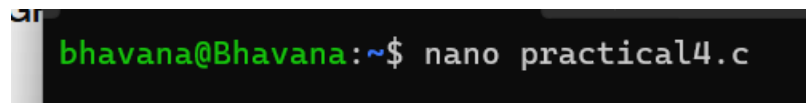
To write a C program in Linux where a parent process creates multiple child processes and synchronizes their execution using `wait()` and `waitpid()`, compare their behavior, create and observe a zombie process using the process table, and eliminate the zombie using proper process synchronization.

Algorithm

1. Start the program.
2. Create multiple child processes using `fork()`.
3. Use `wait()` to wait for any child to finish.
4. Use `waitpid()` to wait for a specific child.
5. Create a zombie by letting the child terminate without `wait()`.
6. Check the process table using `ps` and identify the zombie (Z state).
7. Add `wait()/waitpid()` to the parent and verify that the zombie is removed.
8. Stop the program.

Steps

Step 1

A terminal window with a dark background. The prompt is 'bhavana@Bhavana:~\$' in green. The command 'nano practical4.c' is entered in white text.

```
bhavana@Bhavana:~$ nano practical4.c
```

Step2

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2, child3;
    int status;

    // Create first child
    child1 = fork();

    if (child1 == 0) {
        printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(2);
        printf("Child 1 completed\n");
        exit(10);
    }

    // Create second child
    child2 = fork();

    if (child2 == 0) {
        printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(4);
        printf("Child 2 completed\n");
        exit(20);
    }

    // Create third child
    child3 = fork();

    if (child3 == 0) {
        printf("Child 3: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(1);
        printf("Child 3 completed\n");
        exit(30);
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
**^\
^** Replace

^K Cut
^U Paste

^T Ex
^J Ju

```

        exit(30);
    }

    // Parent waits for any child
    printf("\nParent: Waiting using wait()\n");

    pid_t finished = wait(&status);

    printf("Parent: Child with PID %d finished\n", finished);
    printf("Exit status = %d\n", WEXITSTATUS(status));

    // Parent waits for specific children
    printf("\nParent: Waiting using waitpid()\n");

    waitpid(child1, &status, 0);
    printf("Parent: Child 1 finished\n");

    waitpid(child2, &status, 0);
    printf("Parent: Child 2 finished\n");

    waitpid(child3, &status, 0);
    printf("Parent: Child 3 finished\n");

    printf("\nParent: All children completed.\n");

    return 0;

```

Step3

```

bhavana@Bhavana:~$ gcc practical4.c -o practical4
bhavana@Bhavana:~$ ./practical4

```

```
bhavana@Bhavana:~$ ./practical4
Child 2: PID = 549, PPID = 547

Child 1: PID = 548, PPID = 547
Parent: Waiting using wait()
Child 3: PID = 550, PPID = 547
Child 3 completed
Parent: Child with PID 550 finished
Exit status = 30

Parent: Waiting using waitpid()
Child 1 completed
Parent: Child 1 finished
Child 2 completed
Parent: Child 2 finished
Parent: Child 3 finished

Parent: All children completed.
bhavana@Bhavana:~$ |
```

Step 4 : Create a Zombie Process

```
Parent: All children completed.
bhavana@Bhavana:~$ nano zombie.c
```

Step 5: paste the code

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent is sleeping...\n");

        sleep(30);

        printf("Parent finished.\n");
    }

    return 0;
}

```

Step 6

```

bhavana@Bhavana:~$ gcc zombie.c -o zombie
bhavana@Bhavana:~$ ./zombie

```

```

Parent: PID = 632
Parent is sleeping...
Child: PID = 633
Child is exiting...
Parent finished.

```

The child has already terminated, but the parent is sleeping for 30 seconds.

Therefore, the child becomes a **zombie**.

Step 7 : Eliminate the Zombie using wait()

```
Parent finished.  
bhavana@Bhavana:~$ nano zombie_fixed.c
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else {
        printf("Parent: PID = %d\n", getpid());

        // Wait for child
        wait(&status);

        printf("Parent: Child completed and was collected.\n");
        printf("Parent exiting.\n");
    }

    return 0;
}0|

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J
```

Step 8

```
bhavana@Bhavana:~$ ./zombie_fixed
Parent: PID = 763
Child: PID = 764
Child is exiting...
Parent: Child completed and was collected.
Parent exiting.
bhavana@Bhavana:~$
```

Conclusion

The program demonstrated process creation and synchronization using `wait()` and `waitpid()`. A zombie process was created and identified using `ps`, then eliminated by using proper synchronization in the parent process.